

## 3. Bölüm

# Derleyiciler ve Çalıştırma Ortamı

### 3.1 Derleyici

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler (compiler)** tarafından makine koduna çevrilerek **icra edilebilir (executable)** dosya olarak kaydedilebilmektedir.

C dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları `.c` uzantısı ile dosya olarak saklamaktır. İkinci adım ise **işletim sisteminizde (operating system)** çalışabilen bir icra edilebilir dosya oluşturan bir derleyici kurmaktır. Yani bilgisayarınızda iki yazılım aracına ihtiyacınız vardır; birincisi C derleyicisi, ikincisi ise basit bir metin editörüdür.

Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C **talimatları (statement)** olan, insan tarafından okunabilen metin dosyasıdır. Kaynak kodda verilen talimatlara göre işlemcinizin programı çalıştırabilmesi için, bunun makine diline **derlenmesi (compile)** gerekir.

Derleme, bir başka programlama dilinden **sembolik isim (mnemonic)** ya da **makine koduna (instruction code)** dönüştürme işlemidir.

Bunu yapan programlara **derleyici (compiler)** adı verilir.

### 3.2 C derleyicileri

Günümüzde birçok C derleyicisi mevcuttur. En çok kullanılanları aşağıda verilmiştir;

- ♦ **GNU Derleyici Koleksiyonu (GCC)**: Popüler bir açık kaynaklı C derleyicisidir<sup>1</sup>. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. GCC, çok çeşitli özellikleri ve çeşitli C standartlarına desteğiyle bilinir.
- ♦ **Clang**: LLVM projesinin bir parçası olan açık kaynaklı bir C derleyicisidir<sup>2</sup>. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. Clang, hızı ve optimizasyon yetenekleriyle bilinir.
- ♦ **Microsoft Visual C** : Microsoft tarafından geliştirilen tescilli bir C derleyicisidir<sup>3</sup>. Yalnızca Windows için kullanılabilir. Visual C, Microsoft Visual Studio geliştirme ortamıyla entegrasyonu ile bilinir.

Bilgisayarınızın 64 bitlik Windows olduğu varsayılarak GCC derleyicisi aşağıdaki şekilde kurulur;

- ♦ MINGW sürümü ilgili web sayfasından indirilerek kurulur<sup>4</sup>.
- ♦ Kurulumu tamamlandıktan sonra `PATH` değişkenine GCC derleyicisinin klasörü eklenir.

Böylece kurulum tamamlanmış olur. **Komut istemi (command prompt veya terminal)** açıldığında `gcc -v` komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
ozkan@ILHANOZKAN:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-linux-gnu/13/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgc-ndhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v
--with-pkgversion='Ubuntu 13.3.0-6ubuntu2~24.04.1'
--with-bugurl=file:///usr/share/doc/gcc-13/README.Bugs
--enable-languages=c,ada,c++,go,d,fortran,objc,obj-c++,m2 --prefix=/usr
--with-gcc-major-version-only --program-suffix=-13
```

<sup>1</sup><https://gcc.gnu.org/>

<sup>2</sup><https://clang.llvm.org/>

<sup>3</sup><https://visualstudio.microsoft.com/tr/vs/features/cplusplus/>

<sup>4</sup><https://www.mingw-w64.org/downloads/>

```
--program-prefix=x86_64-linux-gnu- --enable-shared
--enable-linker-build-id --libexecdir=/usr/libexec
--without-included-gettext --enable-threads=posix --libdir=/usr/lib
--enable-nls --enable-bootstrap --enable-clocale=gnu
--enable-libstdcxx-debug --enable-libstdcxx-time=yes
--with-default-libstdcxx-abi=new --enable-libstdcxx-backtrace
--enable-gnu-unique-object --disable-vtable-verify --enable-plugin
--enable-default-pie --with-system-zlib
--enable-libphobos-checking=release --with-target-system-zlib=auto
--enable-objc-gc=auto --enable-multiarch --disable-werror --enable-cet
--with-arch-32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32
--enable-multilib --with-tune=generic
--enable-offload-targets=nvptx-none=/build/gcc-13-EldibY/
gcc-13-13.3.0/debian/tmp-nvptx/usr,amdgc-n-amdhsa=/build/
gcc-13-EldibY/gcc-13-13.3.0/debian/tmp-gcn/usr
--enable-offload-defaulted --without-cuda-driver
--enable-checking=release --build=x86_64-linux-gnu
--host=x86_64-linux-gnu --target=x86_64-linux-gnu
--with-build-config=bootstrap-lto-lean --enable-link-serialization=2
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
ozkan@ILHANOZKAN:~$
```

Kaynak kodumuzu yazmak için bir metin editörü gereklidir. **Windows** işletim sisteminde **notepad**, **Linux** ve **macOs** işletim sistemleri için **nano** programları metin yazmak için idealdir. İlk kaynak kodumuzu oluşturmak için Windows komut isteminde **notepad hello.c**, Linux ve macOS işletim sistemlerinde ise **nano hello.c** komutu yazılır. Editör uygulaması açılır. Eğer böyle bir dosya yoksa metin editörü bu dosyayı oluşturmak için onay ister. Açılan editörü penceresine ilk C programı yazılır ve **hello.c** adlı dosyaya kaydedilir.

```
#include <stdio.h>
```

```
int main() {
    /* İlk C Programı */
    printf("Hello, World! \n");
    return 0;
}
```

Kaydedilen dosyayı komut isteminde **gcc hello.c -o hello.exe** komutu ile derlenir. Böylece hazırlanan kaynak kodumuz derleyici tarafından derlenir (**compile**). Böylece kaynak kodumuz icra edilebilir (**executable**) **hello.exe** dosyasına dönüştürülmüş olur. Derleme ve çalıştırma sürecini **Windows** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
C:\Users\ILHANOZKAN>notepad hello.c
C:\Users\ILHANOZKAN>gcc hello.c -o hello.exe
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

Benzer şekilde derleme ve çalıştırma sürecini **Linux** veya **macOs** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
ozkan@ILHANOZKAN:~$ nano hello.c
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
ozkan@ILHANOZKAN:~$ ./hello.exe
Hello, World!
ozkan@ILHANOZKAN:~$
```

### 3.3 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (**implementation**), derleme (**compile**), icra (**execute**) ya da koşma (**run**) ve izleme (**trace**) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı kısaca **IDE (Integrated Development Environment)** olarak adlandırılır.

IDE'ler kodumuzu renklendirir (**source code coloring** veya **syntax highlighting**) ve kod yazarken daha az hata yapmamızı sağlarlar. Ancak kodu dosyaya yine sade metin olarak kaydederler. Ayrıca kod yazarken

talimatları doğru tamamlamak (**intellisense**) için programcıya yardımcı da olurlar. Bunun gibi kolaylaştırıcı birçok özellikleri bulunur. Örneğin yukarıda yazdığımız düz metin IDE içinde aşağıdaki gibi renkli görünür;

```
#include <stdio.h>
int main() {
    /* İlk C Programı */
    printf("Hello, World! \n");
    return 0;
}
```

C programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- ♦ **Code::Blocks:** Açık kaynaklı bir C/C++ geliştirme ortamı olup bir grafik kullanıcı ara yüzüne **GUI (Graphic User Interface)** sahiptir. Windows, macOS ve Linux işletim sistemleri üzerine kurulabilmektedir.
- ♦ **Visual Studio:** Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Visual Studio, **kod tamamlama (intellisense)**, kullanıcı ara yüzü **UI (user interface)** hazırlama desteği ve **hata ayıklama (debugger)** ve birçok **eklentisi (plug-in)** olan muazzam özelliklere sahiptir.
- ♦ **Visual Studio Code:** Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol (version control)** sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- ♦ **CLion:** JetBrains tarafından geliştirilmiştir ve C++ programcıları için en çok önerilen çapraz platformlu (CMake derleme sistemiyle entegre macOS, Linux ve Windows altında çalışır) geliştirme ortamıdır. Ayrıca, yerel (**local**) ve uzaktan (**remote**) desteğe sahip birkaç IDE'den biri olarak kabul edilir. Bu da yerel bir makinede kod yazmanıza ancak uzak sunucularda derlemenize olanak tanır. Kaynak kodlarımız yönetmemizi sağlayan **CVS (Concurrent Versions System)** ve **TFS (Team Foundation Server)** ile entegre edilebilir.
- ♦ **Eclipse:** Java'da yazılmış ve IBM tarafından geliştirilmiş ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Yaklaşık otuz programlama dilini desteklediği için geniş topluluk desteğiyle iyi bilinir. C++ için Eclipse IDE, kod tamamlama, otomatik kaydetme, derleme ve hata ayıklama desteği, uzak sistem gezgini, statik kod analizi, **profilleme (profiling)** ve **yeniden düzenleme (refactoring)** gibi beklenebilecek tüm özelliklere sahiptir. Ayrıca çeşitli harici eklentileri entegre ederek işlevselliğini genişletebilirsiniz. Windows, Linux ve macOS'ta çalışabilir.
- ♦ **NetBeans:** Apache Yazılım Vakfı – Oracle Corporation tarafından geliştirilen ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Öğrenciler veya başlangıç seviyesindeki C/C++ geliştiricileri için şiddetle tavsiye edilmesinin sebebi, Eclipse'e benzer şekilde daha iyi sürükle ve bırak işlevlerine sahip olmasıdır. Windows, Linux, MacOSX ve Solaris gibi birden fazla platformda çalışır.
- ♦ **Xcode:** MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir geliştirme ortamıdır.

### 3.4 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- ♦ [https://www.tutorialspoint.com/compile\\_c\\_online.php](https://www.tutorialspoint.com/compile_c_online.php)
- ♦ [https://www.onlinegdb.com/online\\_c\\_compiler](https://www.onlinegdb.com/online_c_compiler)
- ♦ <https://www.programiz.com/c-programming/online-compiler/>
- ♦ <https://onecompiler.com/c>
- ♦ [https://www.online-ide.com/online\\_c\\_compiler](https://www.online-ide.com/online_c_compiler)
- ♦ <https://onlinecompilers.com/online-c-compiler>

### 3.5 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı (compile time)** denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır. Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C dilinin **sözdizimi (syntax)** kurallarına uygunluğunu da denetler.

Kod yazımı sırasında dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir **derleme hatası (compile error)** alınarak derleme işlemi sonlandırılacaktır;

```
int main()
{
    return
}
```

Örneğindeki gibi hatalı yazılmış bir kaynak kod aşağıdaki hatayı verecektir;

```
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
hello.c: In function 'main':
hello.c:4:1: error: expected expression before '}' token
4 | }
  | ^
ozkan@ILHANOZKAN:~$
```

Kod yazımı sırasında dilin kurallarına uyulmuş ise en fazla **derleme uyarısı** (**compile warning**) alınacak, ancak derleme tamamlanacaktır;

```
int main()
{
    return;
}
```

Örneğindeki gibi yazılmış bir kaynak kod aşağıdaki uyarıyı verecektir;

```
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
hello.c: In function 'main':
hello.c:3:9: warning: 'return' with no value, in function returning non-void
3 |     return;
  |     ^~~~~~
hello.c:2:5: note: declared here
2 | int main() {
  |     ^~~~
ozkan@ILHANOZKAN:~$
```

Kaynak kod içerisindeki metinlerde Türkçe karakter kullanılması halinde **Windows** altında derlenen programı çalıştırdığınızda aynı metni göremeyebilirsiniz. Bunu düzeltmek için komut isteminde **Chcp 65001** komutu verilerek komut satırının UTF-16 karakter kümesi ile işlem yapmasını sağlayabiliriz.

## 3.6 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları** (**compile time error**): Bu tip hatalar genelde kullanılan dilin yazım kurallarına (**syntax**) uyulmadığından, talimatların (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Koşma zamanı hataları** (**run time error**): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar** (**logical error**): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işlemler (**operator**) kullanılmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken **0!=1** yerine **0!=0** kabul etmek buna örnek verilebilir.

## 3.7 C Programlama Kullanım Alanları

C dili, başlangıçta sistem geliştirme çalışmaları için, özellikle işletim sistemini oluşturan programlar için kullanıldı. **Montaj** (**assembly**) dilinde yazılan kod kadar hızlı çalışan kod ürettiği için sistem geliştirme dili olarak benimsendi. Bilgisayar mühendislerinin bilmesi gereken bu dilin birçok kullanım alanı mevcuttur;

- ◊ İşletim Sistemleri
- ◊ Dil Derleyicileri
- ◊ Assembler Derleyicileri
- ◊ Metin Düzenleyicileri
- ◊ Yazdırma Biriktiricileri
- ◊ Ağ Sürücüler
- ◊ Gömülü Programlar
- ◊ Veri tabanları

### 3.8 C Programlama Dilinin Üstünlükleri

**Verimlilik (efficiency)** ve **hız (speed)**: C, yüksek performanslı ve verimli olmasıyla bilinir. Düşük seviyede bellekle çalışmanıza olanak tanır ve donanımına doğrudan erişim sağlar, bu da onu hız ve ekonomik kaynak kullanımı gerektiren uygulamalar için ideal hale getirir.

**Taşınabilir (portable)**: C programları, minimum veya hiç değişiklik yapılmadan farklı platformlarda derlenebilir ve yürütülebilir. Bu taşınabilirlik, dilin standartlaştırılmış olması ve derleyicilerin küresel olarak çeşitli işletim sistemlerinde kullanılabilmesinden kaynaklanmaktadır.

**Donanım Yakınlık**: C, göstericiler ve düşük seviyeli işlemler kullanılarak donanımın doğrudan manipülasyon yapılmasına olanak tanır. Bu, onu sistem programlama ve donanım kaynakları üzerinde ayrıntılı denetim gerektiren uygulamalar geliştirmek için uygun hale getirir.

**Standart Kütüphaneler**: Giriş/çıkış işlemleri, metin işleme ve matematiksel hesaplamalar gibi yaygın görevler için C, geliştiricilerin önceden oluşturulmuş işlevlerden yararlanarak daha verimli kod yazmalarına yardımcı olan büyük bir standart kütüphaneyle birlikte gelir.

**Yapısal Programlama**: C, kodu modüler ve anlaşılması kolay veri ve kontrol yapılarına sahiptir. Fonksiyonlar, döngüler ve koşullarla geliştiriciler, bakımı kolay net kod üretebilirler.

**Emreden Paradigma**: C dilinde programlama fonksiyonların birbirini çağırması ile yapılır. Bu programlama bakış açısı **yordamlı programlama (procedural programming)** olarak adlandırılır. **Prosedür (procedure)**, bir değer üretmeyen fonksiyonlardır. Fonksiyonlar içinde ise bilgisayarın gerçekleştireceği süreç, basit ve anlaşılır **talimatlar (statement)** halinde art arda yazılır. Talimatların art arda verilmesi **emreden programlama (imperative programming)** olarak da bilinir.

### 3.9 C Dilinin Zayıf Yönleri

**Manuel Bellek Yönetimi**: C dili, bir geliştiricinin belleği açıkça ayırma ve ayırmayı kaldırmasıyla ilgilenmesi gereken manuel bellek yönetimine ihtiyaç duyar. Programcı bunu kodu yazarken yapar.

**Nesne Yönelimli Olmaması**: Günümüzde, programlama dillerinin çoğu **nesne yönelimli programlamayı (object oriented programming)** özelliklerini destekler. Ancak C dili bunu desteklemez.

**Çöp Toplama Olmaması**: C dili, bellekte yer ayrılan ve kullanılmayan bileşenleri bellekten otomatik kaldırmaz. Yani **çöp toplama (garbage collection)** kavramını desteklemez. Programcının belleği manuel olarak ayırması ve kaldırması gerekir ve bu hataya açık olabilir ve bellek sızıntılarına veya verimsiz bellek kullanımına yol açabilir.

**İstisna İşleme Olmaması**: C dili, istisnaları (**exception**) ya da hata ayıklama (**error handling**) işlemek için herhangi bir kütüphane sağlamaz. Programcının her türlü beklentiyi işlemek için kod yazması gerekir.

### 3.10 C Sürümlerinin Tarihçesi

1971 yılında *Dennis Ritchie*, C üzerinde çalışmaya başladı ve Bell Labs'daki diğer geliştiricilerle birlikte C'yi geliştirmeye devam etti. Geleneksel C'den sonraki C sürümlerinin tarihi aşağıda verilmiştir.

**K&R C**, *Dennis Ritchie*, *Brian Kernighan* ile birlikte "The C Programming Language" adlı kitaplarının ilk baskısını 1978 yılında yayınladı. Halk arasında K&R olarak bilinen kitap, uzun yıllar boyunca dilin gayri resmi bir kılavuzu olarak hizmet etti. Açıkladığı C sürümüne genellikle "K&R C" denir ve ayrıca C78 olarak da adlandırılır. K&R C'de tanımlanan C dilinin birçok özelliği, 2018'de onaylanan dilin bir parçasıdır. C'nin erken sürümlerinde, yalnızca **int** dışındaki tipleri döndüren fonksiyonlar, fonksiyon tanımı öncesinde kullanılıyorsa bildirilmelidir; önceden bildirilmeden kullanılan fonksiyonların **int** tipini döndürdüğü varsayılırdı.

**AT&T C** ve diğer satıcılar tarafından üretilen C derleyicileri, K&R C diline eklenen çeşitli özellikleri destekledi. C popülerlik kazanmaya başlasa da farklılıklar problem yaşatmaya başladı. Bu nedenle, dil özelliklerinin standartlaştırılması gerektiği düşünüldü.

**ANSI C**, 1980'lerde, Amerikan Ulusal Standartlar Enstitüsü (ANSI), C dili için resmi bir standart üzerinde çalışmaya başladı. Bu, 1989'da standartlaştırılan ANSI C'nin geliştirilmesine yol açtı. ANSI C, birkaç yeni özellik tanıttı ve dilin önceki sürümlerinde bulunan belirsizlikleri giderdi.

**C89/C90**, ANSI C standardı uluslararası olarak kabul edildi ve C89 (veya onaylanma yılına bağlı olarak C90) olarak tanındı. Uzun yıllar boyunca derleyiciler ve geliştirme araçları için temel teşkil etti.

**C99**, 1999'da ISO/IEC, C99 olarak bilinen C standardının güncellenmiş bir sürümünü onayladı. C standardı 1990'ların sonlarında daha da revize edildi. C99, **satır içi fonksiyonlar (inline function)**, karmaşık sayıları temsil eden karmaşık bir tür gibi çeşitli yeni veri türleri ve değişken uzunluklu diziler vb. gibi yeni özellikler sundu. Ayrıca **//** ile başlayan C++ tarzı tek satırlık yorumlar için destek ekledi.

**C11**, 2011'de yayınlanan C11, C standardının bir başka önemli revizyonudur. C11 standardı, C'ye ve kütüphaneye yeni özellikler ekler ve çoklu iş parçacığı (multi threading) desteği, anonim yapılar (anonym structure) ve birleşimler ve geliştirilmiş Unicode desteği gibi özellikler sunar. Tür genel makroları, anonim yapılar, geliştirilmiş Unicode desteği, atomik işlemler, çoklu iş parçacığı ve sınır denetimli işlevler içerir. C++ ile geliştirilmiş bir uyumluluğu vardır.

**C17**, Bu standart Haziran 2018'de yayınlanmıştır. C17, C programlama dili için geçerli standarttır. Bu standart revizyonuyla yeni özellikler getirilmemiştir. Sadece belirli teknik düzeltmeler ve C11'deki kusurlara yönelik açıklamalar yapar.

**C18**, C standardının en son sürümü olan C18, 2018'de yayınlandı. C11'e kıyasla küçük revizyonlar ve hata düzeltmeleri içeriyor.

**C23**, C23, 2024'te yayınlanan son C dili standardı revizyonunun adıdır. Bu revizyonda 14 yeni anahtar kelime dile eklenmiştir.

C programlama dili, basitliği, verimliliği ve çok yönlülüğü nedeniyle zamanla popülerliğini korudu. İşletim sistemleri, gömülü sistemler, uygulamalar ve oyunlar dahil olmak üzere çeşitli yazılımlar oluşturmak için kullanıldı. C'nin sözdizimi ve semantiği, C++, Java ve Python gibi farklı modern programlama dillerini de etkilemiştir.

### 3.11 Düşün ve Çöz

- ◊ **Düşün:** "Derleme Zamanı Hatası" ile "Çalışma Zamanı Hatası" arasındaki farkı açıklayınız. Hangisinin bir mühendis için tespit edilmesi daha zordur? Neden?
- ◊ **Düşün:** Yorumlayıcı (**interpreter**) diller ile Derleyici (**compiler**) diller arasındaki performans farkının temel sebebi donanım seviyesinde nasıl açıklanır?
- ◊ **Düşün:** Statik bağlama (**static linking**) ve Dinamik bağlama (**dynamic linking**) arasındaki farkları araştırıp, programın dosya boyutu üzerindeki etkisini tartışınız.
- ◊ **Çöz:** Yazdığınız bir C kodunun makine diline dönüşene kadar geçtiği aşamaları (Önişlemci → Derleyici → Bağlayıcı) sırasıyla listeleyiniz.
- ◊ **Çöz:** Bir C programının derleme aşamasında oluşan ara dosyaları ( **.i**, **.s**, **.o** ) ve bu dosyaların içeriklerini gösteren bir şema hazırlayınız.
- ◊ **Çöz:** Sadece bir metin editörü (Notepad vb.) ve bir derleyici (GCC gibi) kullanarak "Merhaba Dünya" programını komut satırından derleyip çalıştırınız.